Comparing Random Forests, MLPs and CNNs for PathMNIST Image Classification

Group 64
Student 1 — SID: 540935875
Student 2 — SID: 550087894
Student 3 — SID: 520271713
Student 4 — SID: 510469531

COMP5318/COMP4318 — Semester 2, 2026

1 Introduction

This study compares a random forest (RF), a multilayer perceptron (MLP), and a convolutional neural network (CNN) on nine-class classification of small colour pathology images. We investigate how their representations and model structures affect predictive performance and computational cost, and how validation-based hyperparameter selection changes their performance. All three models are trained from scratch on the course-provided PathMNIST subset.

Microscope images contain colour and texture patterns that can support tissue classification. This experiment provides a controlled setting for examining how different algorithms use these patterns; it does not establish clinical performance. Comparing algorithms is useful because a higher classification score can come with greater training cost, while a simpler method can provide a practical baseline. The objective is therefore to explain the observed trade-offs, rather than to maximise accuracy with an unrestricted model or search budget.

2 Data

2.1 Data description and exploration

The course data contain 32,000 training images and 8,000 test images, each with dimensions $28 \times 28 \times 3$. Pixel intensities are stored as unsigned 8-bit integers in $[0, 255]$. The assignment identifies these images as a subset of PathMNIST, containing microscope views of normal and abnormal body tissues. PathMNIST is part of the MedMNIST collection described by Yang et al. [1]. We retain the supplied split rather than using the original MedMNIST split. The supplied arrays do not contain a class-name mapping, so this report uses the observed class IDs 0–8 rather than assigning unverified tissue names.

Table 1: Class counts in the supplied data.

Class ID	0	1	2	3	4	5	6	7	8
Training	3490	3431	3505	3656	2950	4290	2728	3253	4697
Test	873	858	877	914	737	1072	682	813	1174

The largest training class accounts for 14.68% of samples and the smallest for 8.53%. This is a moderate imbalance, with approximately 1.72 times as many samples in the largest class as in the smallest. Always predicting the largest training class yields a training majority-class baseline of

14.68%. We report macro F1 alongside accuracy so that smaller classes receive equal weight in one summary measure.

Training-channel means are $(188.65, 135.93, 180.05)$ and standard deviations are $(31.91, 45.60, 32.18)$ for red, green and blue. These statistics show different intensity distributions across channels and motivate retaining colour information. The small image size limits the detail available for classification. Colour similarity between images and variation in local texture are potential challenges: aggregate colour alone may not distinguish images whose spatial arrangements differ. Figure 1 supports inspection of within-class variation and between-class similarities without assuming that each class has a single visual prototype.

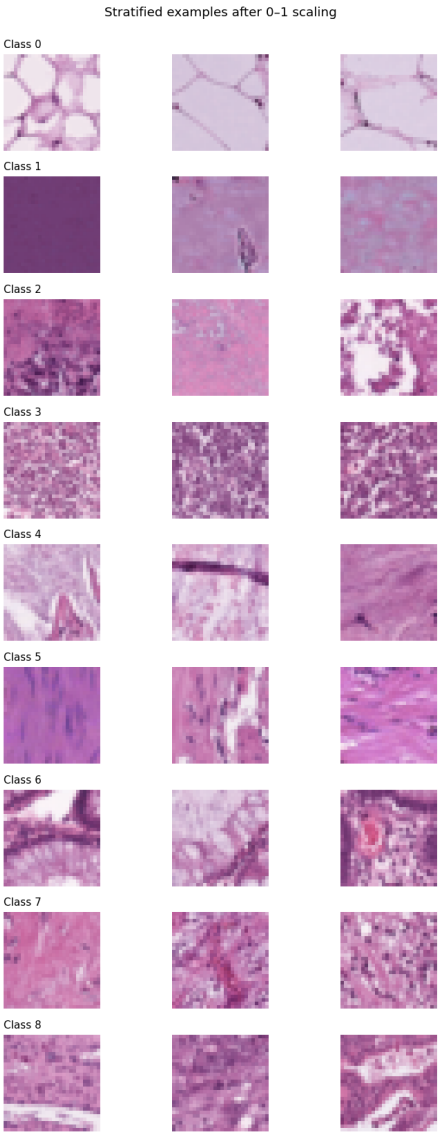


Figure 1: Three training examples from each class after pixel scaling. The sample selection uses a fixed random seed.

2.2 Pre-processing

For both neural networks, pixels are converted to float32 and divided by 255, preserving the image shape while placing inputs in $[0, 1]$. This changes numerical scale without removing colour or spatial information. Labels remain integer class IDs, matching the sparse categorical cross-entropy loss.

For RF, each image is represented by 597 deterministic features. Non-overlapping 2×2 RGB mean pooling produces $14 \times 14 \times 3 = 588$ features. Three channel-wise standard deviations describe colour variability, and six mean absolute adjacent-pixel differences describe horizontal and vertical roughness in the three channels. This reduces the input dimension from 2,352 to 597, retaining coarse location and some texture information while reducing the number of candidate feature values the trees process. The trade-off is loss of fine detail and the limited descriptive power of hand-designed summaries.

These operations are applied independently to each image and fit no statistics on validation or test samples. No augmentation, channel standardisation or further dimensionality reduction is used. Retaining the small native resolution avoids introducing interpolation choices; retaining colour avoids discarding a potentially useful signal. Augmentation could be useful, but its effect is outside this compact comparison and remains a limitation rather than an established unnecessary step.

A stratified 80/20 split of the supplied training set produces 25,600 development images and 6,400 validation images, with random seed 42. All candidates use this same split. The test labels are inspected for descriptive counts but test prediction scores do not guide parameter selection. Once settings are fixed, models are retrained on all 32,000 training images.

3 Methods

3.1 Theory

Random forest. RF aggregates decision trees fitted to bootstrap samples, with random feature subsets considered during splitting [2]. A tree partitions feature space through a sequence of threshold decisions; averaging tree class-probability estimates reduces dependence on an individual tree. Diversity between trees helps control variance, although correlated trees can still make similar errors. This choice connects directly to the Week 5 course topics, Decision Trees and Ensembles: a random forest is an ensemble of decision trees.

Among the methods introduced in the first six weeks, RF offers a suitable baseline for the pooled colour and texture features. Unlike a linear logistic-regression classifier (Week 3), it can represent nonlinear feature interactions without explicitly constructing interaction terms. Unlike Naïve Bayes (Week 4), it does not assume conditional independence between features, which is useful when neighbouring image regions are correlated. Compared with a single decision tree (Week 5), the ensemble aims to reduce variance and sensitivity to the training sample. These properties motivate its selection; they do not establish that it would outperform the other course methods, which were not tested here. RF also provides a useful comparison with MLP and CNN because it learns tree-based decision rules over fixed features rather than a neural representation.

Multilayer perceptron. An MLP transforms the flattened image through learned affine mappings and nonlinear activations. For a hidden layer, the central operation is $h = \mathrm{ReLU}(Wx + b)$, with batch normalisation inserted in this implementation. The softmax output defines probabilities over nine classes. Training minimises

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p(y_i \mid x_i)$$

using Adam [3]. Dense layers can learn interactions among all pixel positions, but do not explicitly encode image locality or reuse the same detector across locations.

Convolutional neural network. A CNN applies shared filters across spatial positions. Local receptive fields and weight sharing are central principles of convolutional image recognition [4].

Early layers learn small patterns, while later layers combine information over larger regions. Weight sharing reduces the number of parameters needed to detect a pattern at multiple positions. Max pooling reduces spatial resolution and provides some tolerance to small changes in feature position, at the cost of detail. The resulting representation is passed to dense layers and a softmax classifier, trained using the same classification loss as the MLP.

3.2 Strengths and weaknesses

RF is fast in this experiment and can represent nonlinear feature interactions. Individual tree decisions are inspectable, although explaining a 120-tree ensemble is harder than explaining a single tree, and feature-importance analysis is not performed here. Its main representational limitation is reliance on fixed summaries: pooling can erase fine structures that the classifier cannot recover.

The MLP learns directly from all pixels and can model complex interactions, but its first dense layer is parameter-heavy. Its treatment of image positions as separate inputs can make learning recurring local patterns less efficient. Batch normalisation supports optimisation [5], and dropout regularises the network by randomly omitting units during training [6], but neither explicitly introduces the spatial structure available to a CNN.

The CNN has an inductive bias suited to local image patterns. Shared filters allow a compact parameterisation, but convolutions can still require many operations at every spatial position. Consequently, fewer parameters do not guarantee faster training. Pooling can discard discriminative information in already small images, and learned filters do not by themselves make the full classifier readily interpretable. These theoretical differences motivate examining both predictive scores and measured runtime.

3.3 Architecture and hyperparameters

The MLP uses Flatten, Dense(H, without bias), batch normalisation, ReLU and dropout, followed by the same sequence with $H/2$ units and a nine-unit softmax layer. Omitting the two hidden-layer biases avoids redundant offsets before batch normalisation. The CNN uses two 3×3 same-padded convolution layers with F and $2F$ filters, each followed by 2×2 max pooling, then Flatten, Dense(64, ReLU), dropout and a nine-unit softmax layer. Both are intentionally small to allow repeated searches.

Table 2: Full factorial search spaces: eight candidates per algorithm.

Model	Hyperparameter	Values and intended effect
RF	Number of trees	60, 120: more averaging, with additional fitting cost
	Maximum depth	12, 20: larger values permit more complex partitions
	Minimum leaf size	1, 3: larger leaves constrain fine-grained fitting
MLP	First hidden-layer width	128, 256: controls capacity and parameter count
	Dropout	0.10, 0.25: controls regularisation strength
	Initial learning rate	0.0005, 0.001: controls optimisation step size
CNN	First-layer filters	16, 24: controls the number of learned feature maps
	Dropout	0.15, 0.35: trades fitting capacity against regularisation
	Learning rate	0.0003, 0.001: controls optimisation step size

A validation-set grid search provides a transparent comparison at manageable cost. Two values per parameter allow selected interactions to be examined without an extensive search; they do not establish a global optimum. Cross-validation would require substantially more neural-network fits. Candidates are ranked by validation accuracy, then macro F1 for exact accuracy ties. RF uses four worker threads. Neural networks use batches of 128, with prediction batches of 256.

MLP search runs for at most 30 epochs, with early stopping after six epochs without validation-accuracy improvement and restoration of the best weights. A validation-loss plateau of three epochs halves its learning rate, down to a floor of 10^{-5}. CNN search runs for at most ten epochs, with early-stopping patience two and no learning-rate reduction. These different budgets make the comparison one of practical configurations rather than equal training opportunities.

Small initial models are fitted before tuning. Initial validation accuracy is 70.47% for RF; the best validation accuracy over five initial epochs is 32.17% for MLP and 66.78% for CNN. Because neural search changes training duration as well as settings, improvement over these initial runs cannot be attributed to hyperparameter changes alone.

4 Results and discussion

4.1 Hyperparameter tuning

Tables 3–5 preserve every candidate's saved validation scores and elapsed fit-plus-validation-prediction time. Neural epoch counts refer to the best validation-accuracy epoch, not necessarily the last epoch executed.

Table 3: RF validation search.

Depth	Min. leaf	Trees	Accuracy (%)	Macro F1	Time (s)
12	1	60	69.52	.6737	3.18
12	1	120	69.52	.6724	6.32
12	3	60	69.52	.6736	3.38
12	3	120	70.30	.6813	6.32
20	1	60	71.17	.6953	4.18
20	1	120	**72.19**	**.7055**	8.35
20	3	60	71.22	.6951	3.97
20	3	120	72.02	.7042	7.92

Increasing RF depth from 12 to 20 improves validation accuracy for every matched tree-count and leaf-size setting. This suggests that the shallower forest restricts useful feature interactions. At depth 20, increasing the tree count from 60 to 120 improves accuracy by 1.02 percentage points with leaf size 1 and 0.80 points with leaf size 3, while approximately doubling time. At depth 12 with leaf size 1, it produces no accuracy improvement. Thus, additional averaging has diminishing value when tree capacity is too restricted. Leaf size has smaller and mixed effects; the results do not justify a universal preference across depths.

Table 4: MLP validation search. H is the first hidden-layer width.

H	Dropout	Initial LR	Accuracy (%)	Macro F1	Best epoch	Time (s)
128	.10	.0005	44.64	.3971	8	7.10
128	.10	.0010	61.31	.5960	30	15.13
256	.10	.0005	64.61	.6271	28	22.04
256	.10	.0010	64.61	.6367	30	21.73
128	.25	.0005	65.41	.6361	29	15.00
128	.25	.0010	**65.81**	**.6384**	29	15.00
256	.25	.0005	62.42	.6089	30	22.20
256	.25	.0010	64.09	.6384	29	21.46

MLP performance depends strongly on interactions between width, dropout and optimisation. At width 128 and dropout .10, increasing the initial learning rate improves accuracy from 44.64%

to 61.31%, but the low-rate run also stops much earlier. This is consistent with sensitivity to the training schedule; it is not proof that learning rate alone explains the difference. With width 128, dropout .25 improves both learning-rate settings. However, at width 256, increasing dropout reduces accuracy at both rates. Stronger regularisation therefore does not consistently help. The wider network also takes roughly 21–22 seconds in its long runs, compared with about 15 seconds for the smaller network, without exceeding the best smaller model's accuracy.

Table 5: CNN validation search. F is the first convolutional layer's filter count.

F	Dropout	LR	Accuracy (%)	Macro F1	Best epoch	Time (s)
16	.15	.0003	70.09	.6953	10	36.03
16	.15	.0010	74.48	.7381	10	36.21
24	.15	.0003	69.14	.6831	10	48.53
24	.15	.0010	**75.56**	**.7442**	9	48.22
16	.35	.0003	68.97	.6754	10	36.33
16	.35	.0010	68.19	.6584	5	25.33
24	.35	.0003	68.50	.6685	10	48.35
24	.35	.0010	74.25	.7256	9	48.41

CNN dropout .15 outperforms .35 for every matched filter-count and learning-rate pair. Within this short budget, stronger dropout may restrict effective fitting. At dropout .15, learning rate .001 outperforms .0003 for both filter counts, consistent with faster progress within ten epochs. Increasing the filter count improves accuracy at the higher rate but slightly reduces it at the lower rate, again showing that extra capacity is not sufficient by itself. Full-length 24-filter runs take around 48 seconds, compared with 36 seconds for 16-filter runs. The relatively short 25.33-second run reflects early stopping and should not be interpreted as an inherently faster architecture.

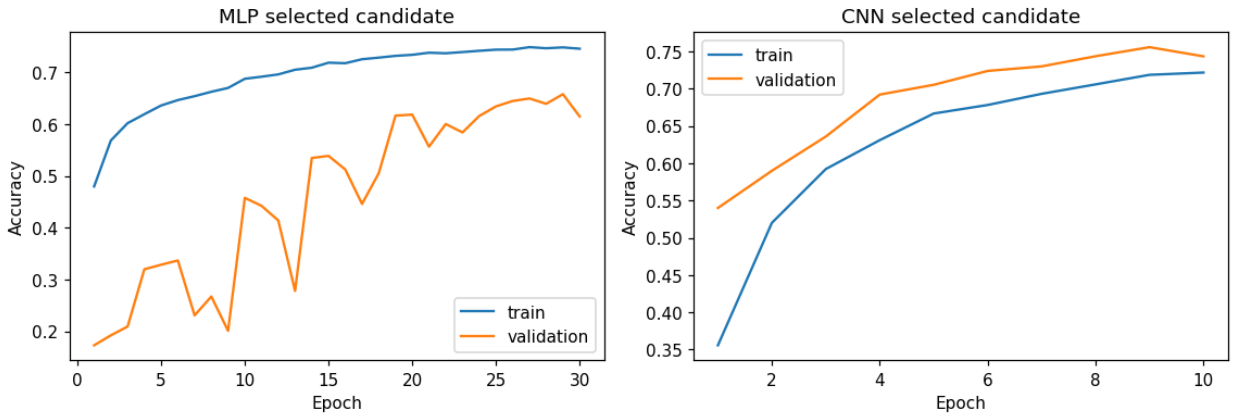


Figure 2: Training and validation accuracy for the selected MLP and CNN search candidates. These curves describe development-set training, not the final full-training-set fits.

The selected MLP reaches its best validation accuracy at epoch 29 of 30; the CNN reaches its best at epoch 9 of 10. Many candidates reach their best near the epoch cap. These observations limit claims that the models have been fully optimised. The curves in Figure 2 show a persistent MLP training–validation gap and substantial early validation fluctuations, consistent with a combination of generalisation and optimisation difficulties. CNN validation accuracy exceeds its recorded training accuracy. This is not necessarily anomalous: dropout is active during training but disabled for validation, and training metrics are accumulated while weights change throughout an epoch. Consequently, the two curves are not identical-condition estimates of generalisation.

4.2 Final model comparison

Final configurations are written explicitly in separate notebook cells, allowing final training without rerunning the searches. RF uses 120 trees, depth 20 and minimum leaf size 1. MLP uses hidden widths 128/64, dropout .25 and initial learning rate .001 for 29 epochs. Its learning rate is halved at the fixed epochs 10, 14, 19 and 24, following the schedule recorded for the selected validation candidate. CNN uses 24/48 filters, dropout .15 and learning rate .001 for nine epochs. The fixed MLP schedule transfers a selected training policy; it does not imply that the full-data optimisation trajectory matches the validation-stage trajectory.

Table 6: Final settings and held-out test results. Timing excludes preprocessing and hyperparameter search.

	RF	MLP	CNN
Best settings	120 trees; depth 20; leaf 1	128/64 units; dropout .25; LR .001; 29 epochs	24/48 filters; dropout .15; LR .001; 9 epochs
Accuracy (%)	72.8125	66.1500	**77.6500**
Macro F1	.7120	.6423	**.7691**
Weighted F1	.7203	.6513	**.7761**
Training time (s)	10.6090	15.0385	49.1028
Prediction time, 8,000 images (s)	.0526	.0428	.4972
Input features / model parameters	597 input features	310,601 parameters	162,265 parameters

CNN exceeds RF accuracy by 4.84 percentage points and MLP accuracy by 11.50 points. Its macro F1 advantage indicates that the gain is not confined to the most frequent class. The result is consistent with the usefulness of learned local representations, although this experiment changes architecture and representation together and cannot isolate either factor causally.

RF is a strong computational baseline: its final training takes about one fifth of the CNN time and exceeds MLP accuracy by 6.66 percentage points. This supports the usefulness of coarse colour and texture summaries on this subset. MLP's greater flexibility does not translate into higher accuracy under the chosen setup, consistent with the difficulty of learning image structure from flattened inputs and the observed sensitivity to optimisation.

CNN has approximately 48% fewer counted parameters than MLP, yet takes 3.27 times as long to train despite fewer epochs. Convolutional filters are reused at many spatial locations, so parameter count is not a direct measure of arithmetic work. MLP prediction is fastest in the saved run; these short measurements are single-run observations, without uncertainty estimates. RF feature count and neural parameter counts describe different quantities and should not be interpreted as a common model-size measure.

Table 7: Selected class-wise test recalls (%).

Model	Class 0	Class 1	Class 2	Class 3	Class 6	Class 7
RF	97.02	93.01	41.85	91.90	42.52	54.74
MLP	94.16	90.21	31.47	85.67	25.81	54.12
CNN	95.07	94.06	49.14	92.01	60.85	63.10

All three models have relatively high recall for classes 0 and 1, whereas classes 2 and 6 are more difficult. CNN improves class-6 recall from RF's 42.52% to 60.85%, but class-2 recall remains below 50%. CNN is not best on every measure: RF has higher class-0 recall. For CNN, class-7 precision is only 51.04% despite recall of 63.10%, indicating a substantial number of false positive assignments to this class. These differences show why a single overall accuracy is insufficient. Confusion patterns suggest areas for targeted investigation, but do not by themselves establish a

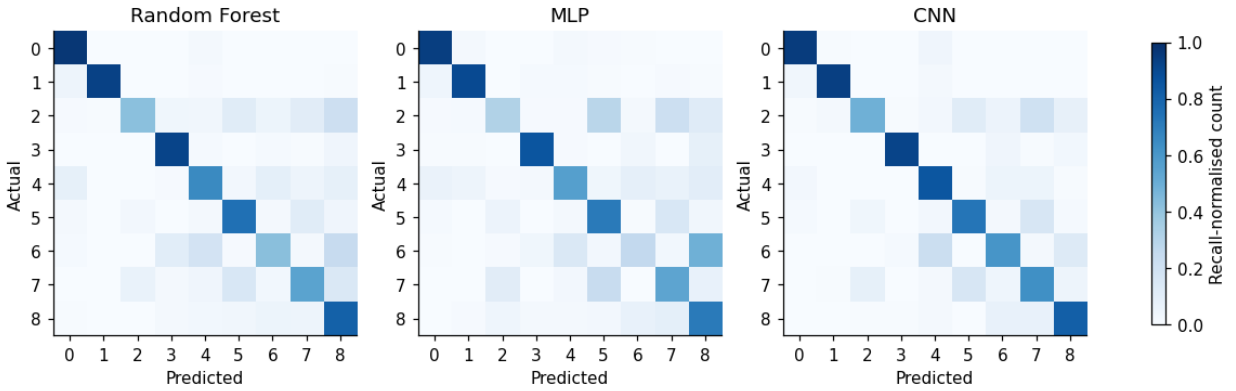


Figure 3: Row-normalised test confusion matrices. Rows are actual classes and columns are predictions; diagonal entries are class recall.

biological explanation.

5 Conclusion

Within this course subset and search budget, CNN provides the strongest predictive performance, achieving 77.65% test accuracy and .7691 macro F1. RF offers a useful speed–performance compromise, while the tested MLP is both less accurate and slower to train than RF. The CNN result is consistent with the benefit of local feature learning and shared weights, but comes with greater measured computational cost. No interpretability or clinical-validation experiment is conducted, so conclusions on those aspects remain limited.

The main limitations are a single validation split and random seed, small search grids, different neural training budgets, and no uncertainty estimates. Architecture and input representation also vary together. The supplied image-level split provides no evidence here about patient-level independence or transfer to other acquisition settings. Several classes remain difficult despite the aggregate CNN improvement.

Future work should repeat the complete validation-based selection over several seeds to assess stability, without using test scores to choose settings. Increasing neural epoch budgets with a consistent stopping policy would test whether near-cap best epochs reflect incomplete training. An RF feature ablation comparing pooled colour alone with the added roughness summaries would quantify their contribution. Finally, modest rotation/flip augmentation, after checking that these transformations preserve labels, could be evaluated on validation data to test robustness to tissue orientation. Each extension addresses a specific uncertainty in the present experiment.

6 Reflection

Student 1 — SID: 540935875

The main lesson I draw from selecting a classical baseline is that an algorithm should be chosen by matching its assumptions to the data. Week 3 logistic regression offers a straightforward linear classifier, while Week 4 Naïve Bayes uses a conditional-independence assumption. Neither property automatically makes these methods unsuitable, but correlated colour and texture features give a reason to consider the nonlinear decision rules of Week 5 trees. Random forest extends this idea by combining diverse trees rather than relying on one partition of the training data. This comparison helps me distinguish a reasoned design choice from a claim of superior performance: our results

support RF as a useful baseline, but cannot show that it is better than logistic regression, Naïve Bayes or SVM because those alternatives were not evaluated. In a future study, I would test one of these alternatives under the same validation protocol to examine the selection argument empirically.

Student 2 — SID: 550087894

The most important lesson I take from the data-processing part of this assignment is that preprocessing determines which information a model can use. Scaling pixels to $[0, 1]$ changes their numerical range while preserving the image, whereas the pooling used for RF removes fine spatial detail. This distinction becomes clear when comparing the neural networks' image inputs with the forest's 597 colour and roughness features. The forest still achieves 72.81% test accuracy, showing that a compact representation can be useful, but this result alone does not prove that pooling improves accuracy. The task also highlights why sample images and class counts should be inspected before training: they provide context for deciding what information to preserve and how to interpret performance. In future work, I would compare feature sets under the same validation conditions rather than judge a preprocessing step only by whether it reduces input size.

Student 3 — SID: 520271713

The main lesson I take from tuning the neural networks is that a final accuracy score needs to be interpreted together with the training process. In the MLP search, increasing dropout helps the 128-unit configurations but reduces accuracy for the 256-unit configurations. This makes it clear that a setting cannot be labelled helpful without considering the rest of the model. The learning curves add another useful insight: the selected MLP has fluctuating validation accuracy early in training, and many candidates reach their best score close to the epoch limit. A weaker result may therefore reflect the optimisation schedule or training budget as well as the architecture. I would now examine learning curves, the best epoch and elapsed time before drawing conclusions from a search table. For a future comparison, I would use a more consistent training-budget policy and repeat runs to check whether the apparent improvements are reliable.

Student 4 — SID: 510469531

The most important lesson I take from analysing the final results is that the best overall score does not describe every aspect of a model's usefulness. CNN achieves 77.65% test accuracy, yet its class-2 recall is only 49.14%. Its class-7 precision is also only 51.04%, showing that identifying many examples of a class does not necessarily mean that predictions of that class are reliable. Reading these scores alongside the confusion matrices gives a more informative picture than ranking models by accuracy alone. Runtime changes the comparison again: RF is less accurate overall but takes substantially less time to train. This task therefore helps me see evaluation as an explanation of trade-offs and failure patterns, rather than simply the selection of a winner. Future improvements should target a clearly identified weakness and be assessed on validation data, without using the final test results to repeatedly adjust the model.

7 AI Acknowledgement

Our learning was grounded in the weekly tutorial code and course concepts. We used OpenAI Codex to help explain and correct code, assist with model implementation and experiment checks, and summarise and interpret selected results. Codex also helped draft and polish the report and format it in LaTeX. The reported numerical results come from the executed notebook.

References

[1] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, vol. 10, article 41, 2023. doi: 10.1038/s41597-022-01721-8.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980, 2014. `https://arxiv.org/abs/1412.6980`.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. `https://bottou.org/papers/lecun-98h`.

[5] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, vol. 37, pp. 448–456, 2015. `https://proceedings.mlr.press/v37/ioffe15.html`.

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. `https://jmlr.org/papers/v15/srivastava14a.html`.